2.3 Les requêtes SQL

Programme

Pédagogie

Compétences attendues

- Écrire des requêtes d'extraction de données en réponse à un besoin d'information
- Écrire des requêtes de mise à jour de données

Savoirs associés

- Requêtes : projection, sélection, jointure, tri, regroupements, restriction sur les groupes, sous-requêtes, utilisation de fonctions
- Requêtes de mise à jour des données de la base

Plan

- **Cours** 2.3.1 Lire une base de données 2.3.2 Construire des requêtes simples 2.3.3 Regrouper, filtrer, trier, assembler 2.3.4 Les requêtes de mise à jour
- **Synthèse**

Mots clés

Clause • Extraction • Fonction • Jointure • Prédicat • Projection • Regroupement • Restriction • Requête • Sélection • Sous-requête • Tri

Le langage SQL (Structured Query Language) permet la définition de tables (création, modification ou suppression), la manipulation de données (extractions, ajouts, modifications ou suppressions de contenus) et leur protection (définition des permissions). Grâce à ce langage, le SI fournit aux différents acteurs d'une organisation des données précises, exhaustives et à jour. Les instructions de traitement formulées en SQL sont des requêtes, autrement dit des demandes de traitement. Comment extraire et mettre à jour les données à l'aide de requêtes SQL ?

À quoi ça sert ?

C'est le chapitre le plus directement employable de tout le programme. Un jour, un dirigeant vous demandera « combien nous ont acheté les clients de la région, l'an dernier ? » — et la réponse ne sera nulle part. Elle existera dans la base, répartie sur quatre tables, et il faudra aller la chercher.

SQL est le langage qui permet de le faire soi-même, sans passer par le service informatique ni attendre trois semaines. Un gestionnaire capable d'écrire une requête n'est plus tributaire des états que quelqu'un d'autre a prévus pour lui.

À l'examen, on vous demandera d'**écrire** des requêtes et d'en **lire**. Ce sont deux exercices différents : lire s'apprend en quelques heures, écrire seulement en écrivant.

Comment travailler ce chapitre

Tout le chapitre interroge **une seule base** : celle de Menuiseries Vasseur, reproduite en entier sur la page **La base de données Menuiseries Vasseur**. Six tables, une soixantaine de lignes.

Chaque requête de ce chapitre est exécutable dans la page. Sous chacune, un bouton **Exécuter** lance la requête pour de bon, et **Modifier** vous laisse la réécrire. Rien à télécharger, rien à installer : le moteur SQL tourne dans votre navigateur.

C'est ainsi qu'il faut lire ce chapitre : **ne croyez aucun résultat, exécutez-le.** Puis modifiez la requête, cassez-la, regardez le message d'erreur, recommencez. On n'apprend pas à écrire du SQL en lisant du SQL.

Le résultat de chaque requête est **aussi imprimé** dans le cours, sous forme de tableau. Il vous sert de corrigé, et il reste là si vous lisez hors ligne ou si le moteur ne se charge pas.

La base vit dans votre navigateur

Vous pouvez tout casser : `UPDATE` sans `WHERE`, `DELETE` de toute une table. **Rien n'est partagé, rien n'est enregistré.** Un bouton « Réinitialiser la base » remet les données d'origine, et recharger la page suffit aussi.

Une seule base sert à tout le chapitre, celle de **Menuiseries Vasseur** : les exemples du cours, les « Votre tour » et les cas pratiques l'interrogent tous.

Cours

2.3.1 Lire une base de données

Chez Vasseur — la situation

Sofiane Belkacem a besoin du chiffre d'affaires réalisé avec les collectivités sur l'exercice. Le logiciel de gestion commerciale propose 40 états préprogrammés ; aucun ne fait exactement cela.

Il demande donc l'extraction à l'éditeur, qui la facture 380 € et l'annonce sous trois semaines. Entre-temps, le conseil d'administration aura eu lieu.

La question de cette section : de quoi une base de données est-elle faite, et quel langage permet de lui poser une question qui n'était pas prévue ?

Le SGBD et le langage SQL

- Le **SGBD** est un intermédiaire entre le programme et les données ;
- Chaque SGBD utilise un langage afin de communiquer avec les applications qui souhaitent accéder aux données.

À savoir

Chaque SGBDR a sa propre manière d'interpréter le SQL. Heureusement, les fonctionnalités de base sont communes : le langage SQL présenté ici s'applique à tous les SGBDR. Mais il arrive que certains SGBDR ne prennent pas en charge certaines fonctionnalités plus avancées, ou bien que la syntaxe du SQL diffère légèrement d'un SGBDR à l'autre.

- Parler en langage SQL, c'est formuler des **requêtes** à la base de données ;
- Une requête peut avoir plusieurs buts : accéder à une donnée, ajouter une donnée, modifier la structure des tables, etc. ;
- Les requêtes se répartissent en **trois groupes** :

Groupe — qui s'en sert	Ce qu'il fait	Mots clés
LDD — langage de définition de données *L'informaticien*	Crée ou supprime des objets dans la base : tables, contraintes	`CREATE`, `ALTER`, `DROP`
LCD — langage de contrôle de données *L'administrateur*	Gère les utilisateurs et leurs droits sur les objets	`GRANT`, `REVOKE`
LMD — langage de manipulation de données ***Le gestionnaire — vous***	Manipule les **lignes** des tables : les créer, les lire, les modifier, les supprimer	`INSERT`, `SELECT`, `UPDATE`, `DELETE`

> ⚠ **Ce que le programme attend de vous**
>
> Le **LMD**, et lui seul. Vous n'aurez ni à créer des tables, ni à distribuer des droits : vous aurez en permanence à **extraire des chiffres**, et parfois à **mettre à jour des données**. C'est la répartition des responsabilités vue en `1.2` — la structure et les droits relèvent de la DSI, l'exploitation des données relève du métier.

> 🔥 **Et le résultat d'une requête ?**
>
> Il s'affiche à l'écran, mais il peut aussi **s'exporter**. Tous les SGBD savent enregistrer un résultat dans un fichier que le tableur ouvrira, le plus souvent au format **CSV** : du texte, une ligne par enregistrement, les valeurs séparées par un point-virgule. C'est le chemin le plus courant entre la base et le tableur : on extrait par une requête, on analyse dans le tableur.
>
> Le fichier exporté est une **photographie** : il ne se met pas à jour quand la base change. Le chapitre 2.6 montre l'autre sens, l'importation dans le tableur, qui peut conserver le lien avec la source.

La base sur laquelle nous allons travailler

- Menuiseries Vasseur a fini par se doter d'une vraie base de données, construite selon les règles des chapitres 2.1 et 2.2 ;

- Elle compte **six tables**, reproduites en entier sur la page **La base de données Menuiseries Vasseur** :

```
CLIENT     (id_client, nom_client, categorie, ville, date_creation)
CHANTIER   (id_chantier, #id_client, ville_chantier, date_debut, date_fin,
montant_ht)
PRODUIT    (ref_produit, designation, matiere, prix_ht)
POSER      (#id_chantier, #ref_produit, quantite)
SALARIE    (id_salarie, nom_salarie, prenom_salarie, fonction, site,
date_embauche)
INTERVENIR (#id_chantier, #id_salarie, date_intervention, nb_heures)
```

> **Le geste à prendre : lire le schéma avant d'écrire**
>
> **On n'interroge pas une base dont on ne connaît pas la structure.** Avant toute requête, on regarde le schéma et on répond à trois questions :
>
> 1. **Où est l'information que je cherche ?** Dans quelle table, dans quelle colonne ?
> 2. **De quelles autres tables ai-je besoin ?** Le nom du client n'est pas dans `CHANTIER` : il faudra aller le chercher ;
> 3. **Par quel chemin les relier ?** Quelle clé étrangère mène à quelle clé primaire ?
>
> Un étudiant qui saute cette étape écrit des requêtes syntaxiquement correctes et logiquement fausses. C'est l'erreur la plus fréquente, et la plus coûteuse en points.

Les clés primaires

> **Rappel**
>
> La **clé primaire** identifie de façon unique et certaine chaque ligne d'une table. Elle ne peut jamais être nulle, et deux lignes ne peuvent pas porter la même valeur.

- Dans la base de Menuiseries Vasseur, les six clés primaires sont :

Table	Clé primaire	Type
`CLIENT`	`id_client`	Simple
`CHANTIER`	`id_chantier`	Simple
`PRODUIT`	`ref_produit`	Simple

Table	Clé primaire	Type
`SALARIE`	`id_salarie`	Simple
`POSER`	(`id_chantier`, `ref_produit`)	**Composée**
`INTERVENIR`	(`id_chantier`, `id_salarie`, `date_intervention`)	**Composée**

> ⚠ **Pourquoi cela vous concerne dès maintenant**
>
> Connaître la clé primaire d'une table permet de **prévoir le nombre de lignes** qu'une requête va renvoyer. C'est le seul garde-fou dont vous disposiez : une requête qui renvoie beaucoup plus de lignes que prévu est une requête fausse, même si elle affiche des chiffres crédibles.
>
> Exemple : `CHANTIER` compte 10 lignes et `id_chantier` en est la clé primaire. Une requête qui part de `CHANTIER` et rapproche `CLIENT` ne peut pas renvoyer plus de 10 lignes. Si elle en renvoie 80, la condition de rapprochement manque.

Mettre en relation plusieurs tables avec des clés étrangères

> ✎ **Rappel**
>
> Une **clé étrangère** est un attribut qui reprend la clé primaire d'une autre table, afin de relier les deux. Elle est notée avec un dièse dans le schéma relationnel.

- La base compte **cinq** clés étrangères :

Clé étrangère	Ce que cela veut dire
`#id_client` dans `CHANTIER` → `CLIENT.id_client`	Chaque chantier appartient à un client
`#id_chantier` dans `POSER` → `CHANTIER.id_chantier`	Chaque ligne de pose concerne un chantier
`#ref_produit` dans `POSER` → `PRODUIT.ref_produit`	…et un produit

Clé étrangère	Ce que cela veut dire
`#id_chantier` dans `INTERVENIR` → `CHANTIER.id_chantier`	Chaque intervention concerne un chantier
`#id_salarie` dans `INTERVENIR` → `SALARIE.id_salarie`	…et un salarié

Une image pour comprendre

Une clé étrangère, c'est le numéro de dossier porté sur un document. Le chantier 105 ne recopie pas l'identité complète de Constructions Ligéria : il porte le numéro 4, et c'est ce numéro qui permet d'aller chercher le reste.

Toute la puissance du modèle tient là : le nom du client est écrit **une seule fois**, et dix chantiers peuvent y renvoyer.

Suivez le chemin à la main

Question : qui est le client du chantier n° 106 ?

1. Dans `CHANTIER`, la ligne 106 porte `id_client = 4` ;
2. Dans `CLIENT`, l'identifiant 4 est **Constructions Ligéria**, promoteur à Tours.

Vous venez d'exécuter une jointure. Le SQL ne fait rien d'autre — il le fait simplement sur 10 000 lignes sans se tromper.

Identifier les types d'associations entre les tables

- Trois types d'associations relient les tables, et **chacun se reconnaît à la place de la clé étrangère** :

Type	Reconnaissance — et dans la base Vasseur
1:1	La clé étrangère est aussi clé primaire de la table fille *Chez Vasseur :* aucun exemple ici — voir l'héritage en 2.2

Type	Reconnaissance — et dans la base Vasseur
1:n	La clé étrangère est un attribut ordinaire de la table « plusieurs » *Chez Vasseur :* `CHANTIER.#id_client` — un client a plusieurs chantiers, un chantier a un seul client
n:n	Une **table de liaison**, dont la clé primaire est composée des deux clés étrangères *Chez Vasseur :* `POSER` entre `CHANTIER` et `PRODUIT` ; `INTERVENIR` entre `CHANTIER` et `SALARIE`

⚠ **Le piège du chemin manquant**

Il n'existe aucun lien direct entre `CLIENT` et `PRODUIT`. Pour répondre à « quels produits Rivière & Fils a-t-il commandés ? », il faut parcourir trois liens :

```
CLIENT ──▶ CHANTIER ──▶ POSER ──▶ PRODUIT
```

Soit **trois jointures**. Les étudiants cherchent souvent une clé étrangère qui n'existe pas, puis renoncent. Le réflexe à installer : **tracer le chemin sur le schéma avant d'écrire la première ligne**.

☰ **Chez Vasseur — la réponse**

De quoi une base de données est-elle faite, et quel langage permet de lui poser une question qui n'était pas prévue ?

Elle est faite de **tables**, reliées par des **clés étrangères** — exactement ce qu'a construit le chapitre 2.1. Ce qui change ici, c'est qu'on ne la conçoit plus : on l'**interroge**.

Le langage est **SQL**, et il se répartit en trois familles : le **LDD** crée la structure, le **LCD** gère les droits, le **LMD** manipule les données. C'est le troisième qui vous concerne, et principalement son verbe `SELECT`.

Pour Sofiane Belkacem, cela signifie qu'il n'a pas besoin de l'éditeur. Son chiffre d'affaires par catégorie de clients tient en **une requête** :

```sql
SELECT     c.categorie, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
GROUP BY   c.categorie ;
```

Vous ne savez pas encore l'écrire — c'est l'objet des deux sections suivantes. Mais vous pouvez déjà en lire la structure : *prendre les chantiers, aller chercher le client de chacun, regrouper par catégorie, additionner les montants*. Les 380 € et les trois semaines payaient l'absence de cette compétence, pas une difficulté technique.

La condition préalable, en revanche, est réelle : **on ne peut pas interroger une base dont on ne connaît pas la structure.** D'où l'ordre de ce chapitre — comprendre les tables et leurs liens avant d'écrire la première requête.

✓ **Vérifiez que vous avez compris**

À quelle famille appartiennent `SELECT`, `CREATE TABLE` et `GRANT` ? Laquelle vous concerne en tant que gestionnaire ?

`SELECT` relève du **LMD** (langage de manipulation de données), `CREATE TABLE` du **LDD** (langage de définition de données), `GRANT` du **LCD** (langage de contrôle de données).

Celle qui vous concerne est le **LMD**. Vous n'aurez ni à créer des tables, ni à distribuer des droits ; vous aurez en permanence à extraire des chiffres et, plus rarement, à corriger des données. C'est aussi la répartition des responsabilités vue en 1.2 : la structure et les droits relèvent de la DSI, l'exploitation des données relève du métier.

Votre tour

Avant de passer aux requêtes, prenez la main sur la base. Modifiez le nom de la table, ajoutez un `WHERE`, faites une faute de frappe volontaire pour voir ce que répond le moteur.

Mise en pratique

Cas pratique 1 : Lire le schéma avant d'écrire

Compétences attendues

- Écrire des requêtes d'extraction de données en réponse à un besoin d'information

Voici le schéma de la base de Menuiseries Vasseur :

```
CLIENT     (id_client, nom_client, categorie, ville, date_creation)
CHANTIER   (id_chantier, #id_client, ville_chantier, date_debut, date_fin,
montant_ht)
PRODUIT    (ref_produit, designation, matiere, prix_ht)
POSER      (#id_chantier, #ref_produit, quantite)
SALARIE    (id_salarie, nom_salarie, prenom_salarie, fonction, site,
date_embauche)
INTERVENIR (#id_chantier, #id_salarie, date_intervention, nb_heures)
```

Missions

1. **Quelle est la clé primaire de `INTERVENIR` ? Que permet-elle, et pourquoi la date en fait-elle partie ?**
2. **Un chantier peut-il être réalisé pour deux clients à la fois, par exemple deux promoteurs associés ? Justifiez à partir du schéma, puis dites ce qu'il faudrait modifier pour le permettre.**
3. **Pour obtenir « le nom de chaque client et la désignation des produits posés sur ses chantiers », combien de tables faut-il rapprocher, et sur quelles conditions ?**
4. **Pour obtenir « le nom des salariés intervenus sur les chantiers de Blois », quelles tables faut-il rapprocher ? Pourquoi la table `CLIENT` est-elle inutile ?**
5. **Le chantier 101 a été posé en septembre 2025. Après la hausse de 4 % des produits en aluminium, la base permet-elle encore de connaître le prix auquel ses fenêtres ont été vendues ? Justifiez à partir du schéma.**

Vérifiez vos réponses

La base est **exécutable** : ses données sont sur la page **La base de données Menuiseries Vasseur**. Écrivez vos requêtes ci-dessous et comparez.

2.3.2 Construire des requêtes simples

Chez Vasseur — la situation

Élodie Fournier veut la liste des chantiers de plus de 50 000 €, avec le **nom** du client — pas son numéro, son nom.

Les montants sont dans la table `CHANTIER`. Les noms sont dans la table `CLIENT`. Les deux ne se rencontrent nulle part sur un écran.

La question de cette section : comment choisir les colonnes, ne garder que les lignes utiles, et faire dialoguer deux tables ?

Afficher les données pertinentes avec SELECT

- `SELECT` répond à la question « **quelles colonnes ?** ». C'est la **projection** ;
- La forme minimale d'une requête comporte deux clauses : `SELECT` (quelles colonnes) et `FROM` (dans quelle table).

Tout afficher — l'étoile

```
1  SELECT * FROM produit ;
```

ref_produit	designation	matiere	prix_ht
F-210	Fenêtre 2 vantaux	aluminium	420,00
F-315	Fenêtre 1 vantail	aluminium	310,00
P-120	Porte-fenêtre	aluminium	690,00
V-400	Véranda 12 m²	aluminium	8 400,00
F-510	Fenêtre 2 vantaux	PVC	265,00

ref_produit	designation	matiere	prix_ht
F-520	Fenêtre 1 vantail	PVC	198,00
P-530	Porte d'entrée	PVC	540,00
C-900	Coulissant 3 rails	aluminium	1 250,00

> ⚠ **L'étoile est commode, pas recommandable**
>
> `SELECT *` sert à **explorer** une table qu'on découvre. Elle n'a pas sa place dans une requête destinée à produire un état :
>
> - elle ramène des colonnes dont on n'a pas besoin, et rend le résultat illisible ;
> - elle transporte des données inutiles sur le réseau — sur une table de 200 000 lignes, la différence est mesurable ;
> - elle **casse silencieusement** le jour où quelqu'un ajoute une colonne : tout ce qui repose sur la position des colonnes se décale.
>
> Sur une copie d'examen, `SELECT *` là où l'énoncé nomme les informations attendues coûte des points.

Choisir les colonnes

```
1  SELECT designation, prix_ht
2  FROM   produit ;
```

designation	prix_ht
Fenêtre 2 vantaux	420,00
Fenêtre 1 vantail	310,00
Porte-fenêtre	690,00
Véranda 12 m²	8 400,00
Fenêtre 2 vantaux	265,00
Fenêtre 1 vantail	198,00

designation	prix_ht
Porte d’entrée	540,00
Coulissant 3 rails	1 250,00

Supprimer les doublons — `DISTINCT`

Le résultat précédent fait apparaître deux fois « Fenêtre 2 vantaux » : le même libellé existe en aluminium et en PVC. Pour obtenir la liste des désignations **distinctes** :

```
SELECT DISTINCT designation
FROM   produit ;
```

designation
Fenêtre 2 vantaux
Fenêtre 1 vantail
Porte-fenêtre
Véranda 12 m²
Porte d’entrée
Coulissant 3 rails

Six lignes au lieu de huit.

> **`DISTINCT` porte sur la ligne entière**
>
> `SELECT DISTINCT designation, matiere FROM produit` renverrait les **huit** lignes, puisque chaque couple (désignation, matière) est unique. `DISTINCT` ne dédoublonne pas colonne par colonne : il dédoublonne le résultat complet. C’est une erreur d’interprétation classique.

Calculer, et renommer avec `AS`

Une requête ne se contente pas de restituer : elle peut **calculer**.

```sql
SELECT designation,
       prix_ht,
       prix_ht * 1.2 AS prix_ttc
FROM   produit
WHERE  matiere = 'PVC' ;
```

designation	prix_ht	prix_ttc
Fenêtre 2 vantaux	265,00	318,00
Fenêtre 1 vantail	198,00	237,60
Porte d'entrée	540,00	648,00

Le mot clé `AS` donne un nom à la colonne calculée. Sans lui, l'en-tête afficherait `prix_ht * 1.2`, ce qui est illisible sur un état remis à un tiers.

> ⚠ **Un calcul dans un `SELECT` ne modifie rien**
>
> La colonne `prix_ttc` n'existe pas dans la base : elle est **calculée à l'affichage**, le temps de la requête. La table `PRODUIT` n'est pas modifiée. C'est la même logique qu'une formule de tableur — et c'est aussi la raison pour laquelle on ne stocke pas les données calculées, comme l'a établi le chapitre 2.1.

Filtrer des lignes avec WHERE

- `WHERE` répond à la question « **quelles lignes ?** ». C'est la **restriction** ;
- La clause suit le `FROM` et porte une **condition**, appelée prédicat, évaluée ligne par ligne.

```sql
SELECT id_chantier, ville_chantier, montant_ht
FROM   chantier
WHERE  montant_ht > 50000 ;
```

id_chantier	ville_chantier	montant_ht
101	Blois	84 500,00
102	Blois	61 200,00
105	Tours	128 400,00

id_chantier	ville_chantier	montant_ht
106	Chambray-lès-Tours	96 750,00

Quatre chantiers sur dix. Vérifiez-le sur la table : c'est bien le cas.

Les opérateurs de comparaison

Opérateur	Signification	Exemple
`=`	Égal	`WHERE categorie = 'promoteur'`
`<>`	Différent	`WHERE ville <> 'Tours'`
`<` `>` `<=` `>=`	Comparaisons	`WHERE montant_ht >= 50000`
`BETWEEN … AND …`	Dans un intervalle, **bornes comprises**	`WHERE montant_ht BETWEEN 10000 AND 50000`
`IN (…)`	Dans une liste	`WHERE ville_chantier IN ('Blois', 'Tours')`
`LIKE`	Ressemble à — `%` remplace n'importe quelle suite de caractères	`WHERE nom_client LIKE 'C%'`
`IS NULL` / `IS NOT NULL`	Vide / non vide	`WHERE date_fin IS NULL`

☰ **Trois filtres, trois résultats**

Les chantiers de Blois ou de Tours :

```
1  SELECT id_chantier, ville_chantier FROM chantier
2  WHERE  ville_chantier IN ('Blois', 'Tours') ;
```

→ six lignes : 101 et 102 (Blois), 104, 105 et 109 (Tours), 110 (Blois).

Les clients dont le nom commence par un C :

```sql
SELECT nom_client FROM client WHERE nom_client LIKE 'C%' ;
```

→ deux lignes : Constructions Ligéria, Collège Balzac.

Les chantiers entre 10 000 et 50 000 € :

```sql
SELECT id_chantier, montant_ht FROM chantier
WHERE  montant_ht BETWEEN 10000 AND 50000 ;
```

→ quatre lignes : 103 (47 800), 107 (12 300), 108 (39 600), 110 (14 750).

Combiner les conditions

`AND` exige que **toutes** les conditions soient vraies, `OR` qu'**au moins une** le soit.

```sql
SELECT nom_client, ville
FROM   client
WHERE  categorie = 'particulier'
   AND ville <> 'Tours' ;
```

nom_client	ville
Nadaud Sylvie	Amboise
Belhadj Karim	Blois

Delaunay Hervé est un particulier, mais il habite Tours : il est écarté par la seconde condition.

⚠ Le cas particulier de la valeur vide

Quatre chantiers n'ont pas de date de fin. Cette absence porte un nom en SQL : `NULL`. Et `NULL` **n'est pas une valeur** : c'est l'absence de valeur.

Conséquence pratique, et elle surprend toujours : `WHERE date_fin = NULL` **ne renvoie rien**. On ne peut pas comparer avec l'absence. Il faut écrire :

```
SELECT id_chantier, ville_chantier, date_debut
FROM   chantier
WHERE  date_fin IS NULL ;
```

id_chantier	ville_chantier	date_debut
102	Blois	2026-01-12
106	Chambray-lès-Tours	2026-02-02
107	Amboise	2026-03-09
109	Tours	2026-04-06

Les quatre chantiers en cours. Retenez la règle : **`IS NULL` et `IS NOT NULL`, jamais `= NULL`.**

Récupérer des données issues de plusieurs tables

La demande d'Élodie Fournier suppose deux tables : les montants sont dans `CHANTIER`, les noms dans `CLIENT`. Que se passe-t-il si l'on nomme simplement les deux ?

```
SELECT * FROM client, chantier ;
```

Le résultat compte **80 lignes** — 8 clients × 10 chantiers. Chaque client a été associé à **tous** les chantiers, y compris ceux qui ne sont pas les siens.

Définition

Le **produit cartésien** associe chaque ligne de la première table à chaque ligne de la seconde. Son nombre de lignes est le **produit** des deux effectifs.

Une image pour comprendre

C'est un plan de table dressé au hasard : on a bien tous les invités et toutes les chaises, mais personne n'est assis à sa place. L'information est là, noyée dans 80 combinaisons dont 70 n'ont aucun sens.

Sur une vraie base, ce n'est pas anecdotique

8 × 10 fait 80. Mais une table de clients de 5 000 lignes et une table de commandes de 40 000 lignes font **200 millions de lignes**. Un produit cartésien involontaire fige le serveur — et c'est la première cause de requête qui « ne rend jamais la main ».

Le garde-fou : **comptez avant de lancer**. Vous connaissez la taille de vos tables ; vous savez donc ce que vous risquez.

Lier des tables avec une jointure interne

Il faut donc **filtrer** ce produit cartésien pour ne garder que les combinaisons cohérentes : celles où l'identifiant du client du chantier est bien celui du client. C'est ce qu'on appelle une **jointure**.

Définition

Une **jointure** est le rapprochement de deux tables selon une **condition de jointure**, qui met le plus souvent en regard la **clé étrangère** de l'une et la **clé primaire** de l'autre.

Techniquement : un produit cartésien, suivi d'un filtrage.

La syntaxe à écrire : `INNER JOIN … ON`

```
SELECT     c.nom_client, ch.id_chantier, ch.ville_chantier, ch.montant_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client ;
```

nom_client	id_chantier	ville_chantier	montant_ht
Rivière & Fils	101	Blois	84 500,00
Rivière & Fils	102	Blois	61 200,00

nom_client	id_chantier	ville_chantier	montant_ht
Mairie de Joué-lès-Tours	103	Joué-lès-Tours	47 800,00
Delaunay Hervé	104	Tours	8 900,00
Constructions Ligéria	105	Tours	128 400,00
Constructions Ligéria	106	Chambray-lès-Tours	96 750,00
Nadaud Sylvie	107	Amboise	12 300,00
Collège Balzac	108	Saint-Avertin	39 600,00
Delaunay Hervé	109	Tours	5 400,00
Belhadj Karim	110	Blois	14 750,00

Dix lignes, et non 80. On est passé du produit cartésien au résultat utile.

Trois points de syntaxe à acquérir tout de suite

- **Les alias** : `chantier ch` et `client c` donnent un nom court à chaque table. Ils rendent la requête lisible dès qu'il y a plus de deux tables ;
- **Le préfixage** : `ch.id_client` et `c.id_client` s'appellent pareil dans les deux tables. Sans préfixe, le SGBD refuse la requête pour ambiguïté ;
- **`ON` porte la condition de rapprochement**, et elle seule. Les conditions de filtrage métier restent dans le `WHERE`.

L'ancienne syntaxe, que vous rencontrerez

La forme `FROM chantier ch, client c WHERE ch.id_client = c.id_client` produit **exactement le même résultat** : c'est l'écriture d'avant la norme **SQL-92**, et elle reste valable dans tous les SGBD.

Vous devez savoir la **lire**, parce qu'une grande partie du code existant l'emploie. Mais ne l'écrivez pas, pour trois raisons :

- elle **mélange** dans le `WHERE` les conditions de jointure et les conditions de filtrage métier, qui n'ont rien à voir ;
- **oublier la condition** ne produit pas une erreur, mais un produit cartésien silencieux ;

- elle **ne sait pas exprimer les jointures externes**, qui n'existent qu'en syntaxe `JOIN … ON`.

Retenez la règle : *le `ON` dit comment les tables se rejoignent, le `WHERE` dit quelles lignes on garde.*

Jointure et filtrage dans la même requête

C'est la demande d'Élodie Fournier :

```sql
SELECT     c.nom_client, ch.id_chantier, ch.montant_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
WHERE      ch.montant_ht > 50000 ;
```

nom_client	id_chantier	montant_ht
Rivière & Fils	101	84 500,00
Rivière & Fils	102	61 200,00
Constructions Ligéria	105	128 400,00
Constructions Ligéria	106	96 750,00

Combien de lignes peut renvoyer une jointure ?

- Chaque chantier possède **au plus un** client — `id_client` est une clé étrangère qui pointe vers une clé primaire, laquelle est unique par définition ;
- Le résultat contient donc **au plus autant de lignes que** `CHANTIER`, soit dix.

> **La règle qui vous sauvera**
>
> Quand on joint une **clé étrangère** d'une table A vers la **clé primaire** d'une table B, le résultat contient **au plus autant de lignes que A**.
>
> Ce simple décompte détecte la quasi-totalité des jointures fausses. Si vous partez de 10 chantiers et que vous en obtenez 80, la condition manque ; si vous en obtenez 4, vous avez perdu des lignes en route.

Joindre trois tables : traverser une table de liaison

Pour savoir ce qui a été posé sur le chantier 105, il faut passer par `POSER`, puis par `PRODUIT` — deux jointures :

```sql
SELECT     ch.id_chantier, p.designation, p.matiere, po.quantite
FROM       chantier ch
INNER JOIN poser po   ON ch.id_chantier = po.id_chantier
INNER JOIN produit p  ON po.ref_produit = p.ref_produit
WHERE      ch.id_chantier = 105 ;
```

id_chantier	designation	matiere	quantite
105	Fenêtre 1 vantail	aluminium	62
105	Fenêtre 2 vantaux	aluminium	30
105	Porte-fenêtre	aluminium	14

Le principe se généralise

Une jointure par lien à traverser. Pour aller de `CLIENT` à `PRODUIT`, il faut trois liens, donc trois `INNER JOIN`. Il n'y a pas d'autre méthode, et il n'y a pas de raccourci.

La bonne pratique consiste à **écrire le chemin en français d'abord** : *client → ses chantiers → ce qui y a été posé → le produit correspondant*. La requête n'est plus que la traduction de cette phrase.

Une jointure sur une clé composée

La clé primaire de `POSER` est le couple (`id_chantier`, `ref_produit`). Une clé étrangère qui la référencerait devrait donc porter **les deux colonnes**, et la condition de jointure les comparer toutes les deux :

```sql
-- non exécutable : t1 et t2 ne sont pas des tables de la base, c'est une forme générale
SELECT *
FROM       t1
INNER JOIN t2 ON t1.fk1 = t2.pk1 AND t1.fk2 = t2.pk2 ;
```

⚠ L'erreur à ne pas commettre

Joindre sur **une seule** des deux colonnes d'une clé composée ne provoque aucune erreur : cela produit un résultat **trop grand**, où chaque ligne est multipliée. Une clé étrangère reprend **l'intégralité** de la clé primaire visée.

Maîtriser les jointures externes

Reprenons la première jointure. Elle renvoie dix lignes — une par chantier. Mais la table `CLIENT` en compte **huit**, et seuls **sept** apparaissent dans le résultat.

SCI Le Clos a disparu. Ce client vient d'être créé et n'a encore aucun chantier : la jointure interne, qui n'associe que les lignes trouvant une correspondance des deux côtés, l'a écarté sans le moindre message.

⚠ C'est le défaut le plus dangereux du chapitre

Une jointure interne ne renvoie pas d'erreur quand elle perd des lignes. Elle renvoie un résultat **plus court**, d'apparence parfaitement normale.

Sur un état « chiffre d'affaires par client », les clients sans commande disparaissent — et personne ne saura qu'ils existent. Sur un état « salariés et leurs interventions », les salariés jamais affectés disparaissent. Ce sont précisément les lignes qu'on cherchait.

✎ Définition

Une **jointure externe** conserve les lignes d'une table même lorsqu'elles n'ont pas de correspondance dans l'autre. Les colonnes manquantes sont alors remplies par `NULL`.

Jointure	Ce qu'elle garde
`INNER JOIN`	Uniquement les lignes appariées des deux côtés
`LEFT OUTER JOIN`	**Toutes** les lignes de la table de gauche, appariées ou non
`RIGHT OUTER JOIN`	**Toutes** les lignes de la table de droite, appariées ou non
`FULL OUTER JOIN`	Toutes les lignes des deux tables

(Le mot `OUTER` est facultatif : `LEFT JOIN` et `LEFT OUTER JOIN` sont identiques.)

Tous les clients, y compris ceux sans chantier

```sql
SELECT      c.nom_client, ch.id_chantier, ch.montant_ht
FROM        client c
LEFT OUTER JOIN chantier ch ON c.id_client = ch.id_client ;
```

nom_client	id_chantier	montant_ht
Rivière & Fils	101	84 500,00
Rivière & Fils	102	61 200,00
Mairie de Joué-lès-Tours	103	47 800,00
Delaunay Hervé	104	8 900,00
Delaunay Hervé	109	5 400,00
Constructions Ligéria	105	128 400,00
Constructions Ligéria	106	96 750,00
Nadaud Sylvie	107	12 300,00
Collège Balzac	108	39 600,00
Belhadj Karim	110	14 750,00
SCI Le Clos	*NULL*	*NULL*

Onze lignes : les dix chantiers, plus SCI Le Clos qui apparaît enfin, avec des cases vides.

Isoler les lignes sans correspondance

En ajoutant un `WHERE` sur la valeur `NULL`, on obtient exactement les orphelins — c'est un usage très fréquent en gestion :

```sql
SELECT      c.nom_client, c.date_creation
FROM        client c
LEFT OUTER JOIN chantier ch ON c.id_client = ch.id_client
WHERE       ch.id_chantier IS NULL ;
```

nom_client	date_creation
SCI Le Clos	2026-01-05

Un seul client n'a jamais donné lieu à un chantier. C'est une information commerciale, pas une anomalie technique : voilà un prospect à relancer.

Le même mécanisme, côté produits

```sql
SELECT          p.ref_produit, p.designation, p.prix_ht
FROM            produit p
LEFT OUTER JOIN poser po ON p.ref_produit = po.ref_produit
WHERE           po.id_chantier IS NULL ;
```

ref_produit	designation	prix_ht
C-900	Coulissant 3 rails	1 250,00

Un produit figure au catalogue et n'a **jamais été posé**. Faut-il le retirer, le promouvoir, revoir son prix ? C'est une question de gestion — mais elle ne se pose que si l'on sait produire la liste, ce qu'une jointure interne n'aurait jamais permis.

> **Le signal à repérer dans un énoncé**
>
> Dès qu'une question contient « **y compris ceux qui…** », « **même si…** », « **tous les…** » ou « **et ceux qui n'en ont pas** », il faut une **jointure externe**. C'est presque un automatisme, et cela vaut plusieurs points à l'examen.

La jointure naturelle

`NATURAL JOIN` effectue la jointure automatiquement sur **toutes les colonnes portant le même nom** dans les deux tables.

```sql
SELECT * FROM chantier NATURAL JOIN client ;
```

> **À connaître, à ne pas employer**
>
> Elle paraît commode, mais elle est **dangereuse** : elle dépend entièrement du nommage des colonnes. Le jour où quelqu'un ajoute à `CLIENT` une colonne `ville_chantier`, ou

renomme une colonne, la jointure change de sens **sans que la requête ait été touchée**.

Sachez la reconnaître si vous la croisez ; écrivez `INNER JOIN … ON`, qui dit explicitement sur quoi porte le rapprochement.

☰ Chez Vasseur — la réponse

Comment choisir les colonnes, ne garder que les lignes utiles, et faire dialoguer deux tables ?

Trois clauses, et une seule requête :

```sql
SELECT     c.nom_client, ch.id_chantier, ch.ville_chantier, ch.montant_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
WHERE      ch.montant_ht > 50000 ;
```

- `SELECT` répond à *quelles colonnes ?* — c'est la **projection**. On nomme les colonnes voulues plutôt que d'écrire `*` ;
- `WHERE` répond à *quelles lignes ?* — c'est la **restriction**, avec `AND`, `OR`, `IN`, `BETWEEN`, `LIKE`, `IS NULL` ;
- `INNER JOIN … ON` répond à *quelles tables, rapprochées comment ?* — c'est la **jointure**, qui met en regard la clé étrangère de l'une et la clé primaire de l'autre.

La demande d'Élodie Fournier tenait en une phrase française ; elle tient en quatre lignes de SQL. Et le mécanisme sous-jacent explique les accidents : une jointure est un **produit cartésien suivi d'un filtrage**. Oubliez la condition, et vous n'obtenez pas une erreur — vous obtenez les 80 combinaisons, silencieusement.

Reste le piège dont il faut se souvenir toute sa vie professionnelle : **`INNER JOIN` fait disparaître ce qui n'a pas de correspondance.** SCI Le Clos n'apparaît pas dans le résultat, et rien ne le signale. Si la question avait été « quel est le chiffre d'affaires de chaque client ? », ce client aurait purement et simplement manqué à l'appel. Les **jointures externes** existent pour cela, et l'énoncé les réclame chaque fois qu'il dit « tous les » ou « y compris ceux qui ».

✓ Vérifiez que vous avez compris

Vous écrivez une jointure entre `CLIENT` (8 lignes) et `CHANTIER` (10 lignes), et le résultat en compte 80. Que s'est-il passé ?

La condition de jointure manque, ou elle est fausse. 8 × 10 = 80 : c'est très exactement le **produit cartésien**, chaque client ayant été associé à tous les chantiers.

Le réflexe à acquérir : devant un résultat anormalement gros, **comptez**. Une jointure d'une clé étrangère vers une clé primaire ne peut pas produire plus de lignes que la table portant la clé étrangère — ici, dix au maximum.

Et devant un résultat anormalement **petit**, comptez aussi : c'est le signe inverse, celui d'une jointure interne qui a écarté des lignes sans correspondance.

Votre tour

Trois exercices à faire ici même, sans quitter la page. Écrivez la requête, exécutez, comparez.

1. la liste des chantiers de Tours, avec leur montant ;
2. le nom du client et la ville de chaque chantier de plus de 40 000 € ;
3. les produits en PVC dont le prix dépasse 250 €.

Mise en pratique

Cas pratique 2 : Ce que la base sait des poseurs

Compétences attendues

- Écrire des requêtes d'extraction de données en réponse à un besoin d'information

Sur la base de Menuiseries Vasseur, écrivez les requêtes répondant aux besoins suivants. **Une requête par question, en syntaxe SQL-92.**

1. **Les salariés du site de Tours : nom, prénom, fonction.**

2. **Les chantiers commencés en 2026 : numéro, ville, date de début, montant.**
3. **Les chantiers situés dans une commune dont le nom contient « Tours » : numéro, ville, montant.** Dites pourquoi `ville_chantier = 'Tours'` ne répond pas à la question.
4. **Pour chaque intervention : nom et prénom du salarié, numéro du chantier, date et nombre d'heures.**
5. **Les interventions réalisées sur les chantiers de Blois : nom du salarié, date et nombre d'heures.**
6. **Les produits en aluminium posés sur les chantiers de Rivière & Fils : numéro du chantier, désignation et quantité.**
7. **Question de réflexion : la requête 4 fait-elle apparaître les salariés qui ne sont jamais intervenus sur un chantier ? Justifiez, et écrivez la requête qui les fait apparaître.**

Vérifiez vos réponses

La base est **exécutable** : ses données sont sur la page **La base de données Menuiseries Vasseur**. Écrivez vos requêtes ci-dessous et comparez.

2.3.3 Regrouper, filtrer, trier, assembler

Chez Vasseur — la situation

Camille Vasseur ne veut pas la liste des dix chantiers. Elle veut **un chiffre par catégorie de clients** : combien avec les particuliers, combien avec les promoteurs, combien avec les collectivités — et seulement les catégories qui dépassent 50 000 €.

Toutes les requêtes vues jusqu'ici renvoient des lignes de détail. Aucune ne sait compter.

La question de cette section : comment passer du détail au chiffre agrégé, et comment filtrer sur ce chiffre agrégé ?

Agréger des lignes avec GROUP BY

Définition

Une **fonction d'agrégation** calcule une valeur unique à partir d'un ensemble de lignes. `GROUP BY` définit les **groupes** sur lesquels ce calcul s'effectue.

Fonction	Ce qu'elle calcule
`COUNT(*)`	Le nombre de lignes du groupe
`SUM(colonne)`	La somme
`AVG(colonne)`	La moyenne
`MIN(colonne)` / `MAX(colonne)`	Le plus petit / le plus grand

Sans regroupement : un seul chiffre pour toute la table

```sql
SELECT COUNT(*)     AS nb_chantiers,
       SUM(montant_ht) AS ca_total,
       AVG(montant_ht) AS montant_moyen,
       MAX(montant_ht) AS plus_gros_chantier
FROM   chantier ;
```

nb_chantiers	ca_total	montant_moyen	plus_gros_chantier
10	499 600,00	49 960,00	128 400,00

Avec regroupement : un chiffre par groupe

```sql
SELECT     c.categorie,
           COUNT(*)            AS nb_chantiers,
           SUM(ch.montant_ht)  AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
GROUP BY   c.categorie ;
```

categorie	nb_chantiers	ca_ht
collectivité	2	87 400,00
particulier	4	41 350,00
promoteur	4	370 850,00

Vérifiez-le à la main — c'est le meilleur moyen de comprendre

Les promoteurs sont Rivière & Fils (chantiers 101 et 102) et Constructions Ligéria (105 et 106) : 84 500 + 61 200 + 128 400 + 96 750 = **370 850 €**.

Et le contrôle qui doit devenir un réflexe : 87 400 + 41 350 + 370 850 = **499 600 €**, soit exactement le total de la table. Si la somme des groupes ne fait pas le total, un groupe a été perdu — par exemple parce qu'une jointure interne a écarté des lignes.

Regrouper sans jointure

```
SELECT   ref_produit, SUM(quantite) AS total_pose
FROM     poser
GROUP BY ref_produit ;
```

ref_produit	total_pose
F-210	101
F-315	62
F-510	113
F-520	28
P-120	20
P-530	15
V-400	2

Sept lignes, et non huit : `C-900` **est absent**, puisqu'il n'apparaît dans aucune ligne de `POSER`. Le même piège qu'en 2.3.2 — et la même solution, la jointure externe, si l'on veut le voir figurer avec un total de zéro.

La règle du `SELECT` avec `GROUP BY`

Dans une requête comportant un `GROUP BY`, le `SELECT` ne peut contenir que :

- les **colonnes de regroupement** — celles qui figurent dans le `GROUP BY` ;
- des **fonctions d'agrégation**.

Écrire `SELECT c.categorie, c.nom_client, SUM(ch.montant_ht) … GROUP BY c.categorie` n'a **aucun sens** : la catégorie « promoteur » regroupe deux clients différents, quel nom faudrait-il afficher ? Certains SGBD refusent la requête, d'autres en choisissent un au hasard — ce qui est pire, car le résultat paraît correct.

Le test : *« cette colonne a-t-elle une valeur unique pour tout le groupe ? »* Si non, elle n'a rien à faire dans le `SELECT`.

Regrouper sur plusieurs colonnes

```sql
SELECT     s.nom_salarie,
           COUNT(*)          AS nb_interventions,
           SUM(i.nb_heures)  AS total_heures
FROM       intervenir i
INNER JOIN salarie s ON i.id_salarie = s.id_salarie
GROUP BY   s.nom_salarie ;
```

nom_salarie	nb_interventions	total_heures
Aubry	6	37,5
Mercier	6	44,0

Deux poseurs, douze interventions, 81,5 heures au total.

Ce que cette requête révèle du modèle

Elle n'est possible que parce que la date fait partie de la clé primaire de `INTERVENIR`. Si la table ne contenait qu'une ligne par couple (chantier, salarié), les deux journées consécutives de Yann Mercier sur le chantier 101 auraient été réduites à une seule, et le total d'heures serait faux.

La modélisation détermine les questions qu'on pourra poser. C'est la leçon du chapitre 2.2, et elle se vérifie ici en chiffres.

Filtrer les données agrégées avec HAVING

C'est la demande de Camille Vasseur : ne garder que les catégories dépassant 50 000 €.

Le premier réflexe — ajouter un `WHERE` — ne fonctionne pas :

```sql
-- REQUÊTE FAUSSE
SELECT     c.categorie, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
WHERE      SUM(ch.montant_ht) > 50000
GROUP BY   c.categorie ;
```

Le SGBD la refuse. La raison est logique : **au moment où le `WHERE` s'exécute, aucune somme n'a encore été calculée.** Le `WHERE` travaille ligne par ligne, avant tout regroupement.

> **Définition**
>
> `HAVING` filtre les **groupes**, après le regroupement, comme `WHERE` filtre les lignes avant.

```sql
SELECT     c.categorie, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
GROUP BY   c.categorie
HAVING     SUM(ch.montant_ht) > 50000 ;
```

categorie	ca_ht
collectivité	87 400,00
promoteur	370 850,00

Les particuliers (41 350 €) sont écartés.

> ⚠ **`WHERE` et `HAVING` — la distinction la plus rentable du chapitre**

	`WHERE`	`HAVING`
Filtre	des **lignes de détail**	des **groupes**
S'exécute	**avant** le regroupement	**après** le regroupement

	`WHERE`	`HAVING`
Peut porter sur une fonction d'agrégation ?	**Non**	**Oui — c'est sa raison d'être**

La règle opérationnelle : **toute condition portant sur un `SUM`, `COUNT`, `AVG`, `MIN` ou `MAX` va dans le `HAVING`. Toute autre condition va dans le `WHERE`.**

Les deux dans la même requête

```sql
SELECT     c.categorie,
           COUNT(*)           AS nb_chantiers,
           SUM(ch.montant_ht) AS ca_termine
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
WHERE      ch.date_fin IS NOT NULL
GROUP BY   c.categorie
HAVING     SUM(ch.montant_ht) > 50000 ;
```

categorie	nb_chantiers	ca_termine
collectivité	2	87 400,00
promoteur	2	212 900,00

Lecture de la requête, dans l'ordre où le SGBD l'exécute :

1. on rapproche chantiers et clients ;
2. `WHERE` : on écarte les chantiers non terminés — il en reste six ;
3. `GROUP BY` : on regroupe par catégorie ;
4. `HAVING` : on écarte les catégories sous 50 000 € — les particuliers, à 23 650 €.

Pourquoi l'ordre compte

Intervertir les deux conditions ne donnerait pas un résultat approximatif : cela donnerait un résultat **faux**, ou une erreur. Ici, écarter les chantiers en cours **avant** de sommer est indispensable — sinon les 370 850 € des promoteurs incluraient des chantiers non achevés, donc du chiffre d'affaires non acquis.

C'est une question comptable autant que technique.

Ordonner les résultats avec ORDER BY

- `ORDER BY` trie le résultat. Elle est **toujours la dernière clause** de la requête ;
- `ASC` (croissant) est implicite, `DESC` donne l'ordre décroissant.

```
SELECT   id_chantier, ville_chantier, montant_ht
FROM     chantier
ORDER BY montant_ht DESC ;
```

id_chantier	ville_chantier	montant_ht
105	Tours	128 400,00
106	Chambray-lès-Tours	96 750,00
101	Blois	84 500,00
102	Blois	61 200,00
103	Joué-lès-Tours	47 800,00
108	Saint-Avertin	39 600,00
110	Blois	14 750,00
107	Amboise	12 300,00
104	Tours	8 900,00
109	Tours	5 400,00

Trier sur plusieurs colonnes

```
SELECT   nom_client, ville, categorie
FROM     client
ORDER BY ville, nom_client ;
```

nom_client	ville	categorie
Nadaud Sylvie	Amboise	particulier

nom_client	ville	categorie
Belhadj Karim	Blois	particulier
Rivière & Fils	Blois	promoteur
Mairie de Joué-lès-Tours	Joué-lès-Tours	collectivité
Collège Balzac	Saint-Avertin	collectivité
Constructions Ligéria	Tours	promoteur
Delaunay Hervé	Tours	particulier
SCI Le Clos	Vendôme	promoteur

Le tri s'effectue **d'abord** par ville, **puis**, à ville égale, par nom : c'est ce qui range Belhadj Karim avant Rivière & Fils dans Blois.

Trier sur une colonne calculée

On peut trier sur l'alias défini dans le `SELECT` :

```
SELECT     c.categorie, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
GROUP BY   c.categorie
ORDER BY   ca_ht DESC ;
```

categorie	ca_ht
promoteur	370 850,00
collectivité	87 400,00
particulier	41 350,00

C'est exactement le tableau que demandait Camille Vasseur, dans l'ordre où elle veut le lire.

Assembler des résultats avec UNION

Définition

`UNION` empile les lignes de deux requêtes l'une sous l'autre. Les deux requêtes doivent renvoyer le **même nombre de colonnes**, de **types compatibles**.

Menuiseries Vasseur veut la liste de **toutes les communes où elle a une présence** — soit parce qu'un client y est domicilié, soit parce qu'un chantier s'y trouve.

```
1  SELECT ville           FROM client
2  UNION
3  SELECT ville_chantier  FROM chantier ;
```

ville
Amboise
Blois
Chambray-lès-Tours
Joué-lès-Tours
Saint-Avertin
Tours
Vendôme

Sept communes. Notez que Chambray-lès-Tours n'apparaît que côté chantiers (aucun client n'y est domicilié), et Vendôme que côté clients (SCI Le Clos n'a pas encore de chantier).

⚠ `UNION` supprime les doublons, `UNION ALL` les conserve

Les deux requêtes renvoient 8 et 10 lignes, soit 18 au total. `UNION` n'en restitue que **7**, parce qu'elle **dédoublonne**.

`UNION ALL` restituerait les **18 lignes**, Blois apparaissant cinq fois. C'est plus rapide — le SGBD n'a rien à comparer — et c'est ce qu'il faut employer quand on sait qu'il n'y a pas de doublon, ou quand on veut les conserver.

⚠ `UNION` n'est pas une jointure

Confusion très fréquente, et sanctionnée :

- une **jointure** ajoute des **colonnes** — elle met deux tables côte à côte ;
- une `UNION` ajoute des **lignes** — elle empile deux résultats.

Le test : *« ai-je besoin de plus d'informations sur chaque ligne, ou de plus de lignes ? »*

Imbriquer une requête dans une autre : les sous-requêtes

Camille Vasseur pose une question simple : *« quels chantiers dépassent notre montant moyen ? »* Le réflexe consiste à écrire `WHERE montant_ht > AVG(montant_ht)`, et le SGBD refuse, pour la raison déjà vue avec `HAVING` : une fonction d'agrégation n'a pas sa place dans un `WHERE`.

La solution consiste à faire calculer la moyenne **d'abord**, par une requête à part, puis à utiliser son résultat dans la requête principale.

Définition

Une **sous-requête** est une requête `SELECT` écrite **entre parenthèses** à l'intérieur d'une autre requête. Le SGBD l'exécute en premier, puis se sert de son résultat comme d'une valeur ou d'une liste.

Une sous-requête qui renvoie une seule valeur

```
SELECT   id_chantier, ville_chantier, montant_ht
FROM     chantier
WHERE    montant_ht > (SELECT AVG(montant_ht) FROM chantier)
ORDER BY montant_ht DESC ;
```

id_chantier	ville_chantier	montant_ht
105	Tours	128 400,00
106	Chambray-lès-Tours	96 750,00
101	Blois	84 500,00

id_chantier	ville_chantier	montant_ht
102	Blois	61 200,00

La sous-requête renvoie **49 960** : 499 600 € répartis sur dix chantiers. La requête principale compare chaque chantier à ce nombre. Exécutez la sous-requête seule : c'est la meilleure façon de vérifier ce qu'elle renvoie.

Une sous-requête de ce type s'emploie derrière un opérateur de comparaison : `=`, `<>`, `>`, `<`, `>=`, `<=`.

Une sous-requête qui renvoie une liste : `IN`

Thomas Lemoine doit prévenir les clients dont un chantier est **en cours**. Les chantiers en cours sont dans `CHANTIER`, les noms dans `CLIENT` :

```
SELECT   nom_client, categorie
FROM     client
WHERE    id_client IN (SELECT id_client
                       FROM   chantier
                       WHERE  date_fin IS NULL)
ORDER BY nom_client ;
```

nom_client	categorie
Constructions Ligéria	promoteur
Delaunay Hervé	particulier
Nadaud Sylvie	particulier
Rivière & Fils	promoteur

La sous-requête renvoie la liste des clients concernés ; `IN` garde ceux dont l'identifiant figure dans cette liste. C'est le même `IN` que dans `WHERE ville_chantier IN ('Blois', 'Tours')`, à ceci près que la liste est **calculée** au lieu d'être écrite à la main.

Ce qui ne figure pas dans la liste : `NOT IN`

```
SELECT ref_produit, designation, matiere
FROM   produit
WHERE  ref_produit NOT IN (SELECT ref_produit FROM poser) ;
```

ref_produit	designation	matiere
C-900	Coulissant 3 rails	aluminium

Vous avez déjà obtenu ce coulissant jamais posé avec une **jointure externe** (`LEFT OUTER JOIN … WHERE po.id_chantier IS NULL`). Les deux écritures sont justes. La sous-requête est souvent la plus simple à écrire et à relire.

⚠ Une valeur ou une liste ?

- Derrière `=`, `>`, `<`…, la sous-requête doit renvoyer **une seule valeur**. Si elle en renvoie plusieurs, la plupart des SGBD refusent la requête.
- **SQLite, le moteur de la console, ne la refuse pas** : il prend la première valeur et ne dit rien. `WHERE id_client = (SELECT id_client FROM chantier WHERE date_fin IS NULL)` ne renvoie que Rivière & Fils, alors que quatre clients sont concernés. C'est plus dangereux qu'une erreur.
- Dès que la sous-requête peut renvoyer plusieurs lignes, employez `IN` ou `NOT IN`.

La méthode

1. Écrivez et exécutez d'abord la sous-requête **seule**, et vérifiez son résultat.
2. Placez-la entre parenthèses, derrière `IN` ou derrière un opérateur de comparaison.
3. Relisez le besoin : « les … qui figurent parmi … » appelle un `IN` ; « au-dessus de la moyenne », « le plus cher » appellent une comparaison à une sous-requête qui calcule un agrégat.

✓ Vérifiez que vous avez compris

« Les clients qui n'ont encore jamais eu de chantier. » Écrivez la requête avec une sous-requête.

```sql
SELECT nom_client
FROM   client
WHERE  id_client NOT IN (SELECT id_client FROM chantier) ;
```

Un seul client : **SCI Le Clos**, créé en janvier 2026. `NOT IN` garde les clients dont l'identifiant n'apparaît **nulle part** dans la table des chantiers ; `IN` aurait donné les sept autres.

☰ Chez Vasseur — la réponse

Comment passer du détail au chiffre agrégé, et comment filtrer sur ce chiffre agrégé ?

Par `GROUP BY`, et par la clause qui lui est propre, `HAVING` :

```sql
SELECT     c.categorie, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
GROUP BY   c.categorie
HAVING     SUM(ch.montant_ht) > 50000
ORDER BY   ca_ht DESC ;
```

Camille Vasseur obtient son tableau : promoteurs 370 850 €, collectivités 87 400 €, les particuliers écartés à 41 350 €.

Le seul vrai obstacle du chapitre tient en une phrase : **`WHERE` et `HAVING` ne filtrent pas la même chose.** Le premier écarte des **lignes de détail** avant le regroupement ; le second écarte des **groupes** après. Une condition portant sur une fonction d'agrégation va toujours dans le `HAVING` ; toute autre condition va dans le `WHERE`.

Second point de vigilance : dans une requête avec `GROUP BY`, le `SELECT` ne peut contenir que **les colonnes de regroupement et des fonctions d'agrégation**. Demander le nom d'un client à côté d'une somme par catégorie n'a pas de sens.

Et l'ordre d'écriture des clauses, à retenir tel quel :

```
SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY
```

✓ Vérifiez que vous avez compris

« Le nombre de chantiers par ville, pour les seules villes en comptant plus de deux, et uniquement les chantiers terminés. » Où va chacune des deux conditions ?

- « chantiers terminés » porte sur **une ligne de détail** — chaque chantier a, ou n'a pas, une date de fin. Elle va dans le `WHERE`, avant regroupement ;
- « plus de deux chantiers » porte sur **le groupe** — c'est un `COUNT`. Elle va dans le `HAVING`, après regroupement.

```sql
SELECT   ville_chantier, COUNT(*) AS nb
FROM     chantier
WHERE    date_fin IS NOT NULL
GROUP BY ville_chantier
HAVING   COUNT(*) > 2 ;
```

Sur cette base, le résultat est **vide** : aucune ville ne compte plus de deux chantiers terminés. Un résultat vide n'est pas une erreur — c'est une réponse, et il faut savoir le dire plutôt que de « corriger » la requête jusqu'à ce qu'elle rende quelque chose.

Notez enfin l'ordre des opérations : on écarte d'abord les chantiers en cours, **puis** on compte. L'inverse donnerait un tout autre résultat.

Votre tour

1. le nombre de chantiers par ville ;
2. le montant total posé par produit, du plus vendu au moins vendu ;
3. les villes où l'entreprise a réalisé plus de 50 000 € de chantiers ;
4. les produits plus chers que le prix moyen du catalogue.

Mise en pratique

Cas pratique 3 : Le tableau de bord des chantiers

Compétences attendues

- Écrire des requêtes d'extraction de données en réponse à un besoin d'information

Camille Vasseur prépare la réunion trimestrielle. Sur la base de Menuiseries Vasseur, écrivez les requêtes répondant à ses questions.

1. **Le nombre total d'heures de pose par chantier.**
2. **Pour chaque client : son nom, son nombre de chantiers et le montant total de ses chantiers, du plus gros client au plus petit.**
3. **Les chantiers ayant mobilisé plus de 12 heures de pose.**
4. **Le nombre de fenêtres posées, par matière.**
5. **Le montant moyen des chantiers par ville, pour les seules villes comptant au moins deux chantiers.**
6. **Les salariés qui ne sont jamais intervenus sur un chantier, en utilisant une sous-requête.** Comparez avec la question 7 du cas 2.3-2.
7. **Les chantiers de particuliers dont le montant dépasse le montant moyen des chantiers de particuliers.**
8. **Question de réflexion : pour la requête 3, pourquoi la condition « plus de 12 heures » ne peut-elle pas s'écrire dans le `WHERE` ?**

Vérifiez vos réponses

La base est **exécutable** : ses données sont sur la page **La base de données Menuiseries Vasseur**. Écrivez vos requêtes ci-dessous et comparez.

2.3.4 Les requêtes de mise à jour

Chez Vasseur — la situation

Le tarif du profilé aluminium augmente de 4 % au 1er avril. Toutes les références en aluminium sont concernées.

Nadia Perrot a commencé à les modifier une par une dans le logiciel. Sur les 1 340 références du catalogue réel, elle en est à la trentième, il est 19 h, et elle n'est pas sûre de ne pas en avoir sauté une.

La question de cette section : comment modifie-t-on des données en masse, et comment ne pas détruire la base en le faisant ?

- Les trois sections précédentes n'ont fait que **lire** la base. Or le programme attend aussi que vous sachiez écrire des **requêtes de mise à jour** ;
- Elles appartiennent au **LMD**, comme `SELECT`, et se réduisent à trois verbes :

Verbe	Ce qu'il fait	Effet sur le nombre de lignes
`INSERT INTO`	Ajoute une ou plusieurs lignes	Augmente
`UPDATE`	Modifie la valeur de colonnes sur des lignes existantes	Inchangé
`DELETE FROM`	Supprime des lignes	Diminue

> ⚠ **Avant toute chose**
>
> Ces trois requêtes **modifient la base**. Contrairement à un `SELECT`, elles ne sont pas rejouables sans conséquence, et il n'y a pas d'annulation.
>
> **La règle professionnelle, à appliquer sans exception : écrivez d'abord la requête en `SELECT`.** Si le `SELECT` renvoie exactement les lignes que vous vouliez toucher, remplacez-le par `UPDATE` ou `DELETE`. C'est la seule protection réellement efficace, et elle ne coûte que dix secondes.

Ajouter des lignes : INSERT INTO

```
1  INSERT INTO client (id_client, nom_client, categorie, ville, date_creation)
2  VALUES (9, 'Résidence Les Tilleuls', 'promoteur', 'Tours', '2026-04-02') ;
```

La table `CLIENT` compte désormais neuf lignes.

- On peut insérer **plusieurs lignes** en une seule requête, en séparant les jeux de valeurs par des virgules :

```sql
INSERT INTO produit (ref_produit, designation, matiere, prix_ht)
VALUES ('F-540', 'Fenêtre 3 vantaux', 'PVC',       395.00),
       ('P-140', 'Porte-fenêtre 3 vantaux', 'aluminium', 980.00) ;
```

Trois points de vigilance

- **Nommez les colonnes.** `INSERT INTO client VALUES (…)` fonctionne, mais dépend de l'ordre physique des colonnes : le jour où quelqu'un en ajoute une, toutes vos insertions deviennent fausses sans erreur visible ;
- **Les colonnes non citées** prennent leur valeur par défaut, ou `NULL`. Si la colonne est déclarée `NOT NULL` sans valeur par défaut, la requête est rejetée ;
- **La clé primaire doit être unique** : insérer un `id_client` déjà présent provoque un rejet. C'est une protection, pas un obstacle.

L'intégrité référentielle, dans l'autre sens

Insérer un chantier rattaché à un client inexistant est **refusé** par le SGBD :

```sql
INSERT INTO chantier (id_chantier, id_client, ville_chantier, date_debut, montant_ht)
VALUES (111, 47, 'Tours', '2026-05-04', 22000.00) ;   -- le client 47 n'existe pas
```

C'est exactement ce qu'on attend d'une base de données, et ce qu'un tableur ne sait pas faire : **elle refuse les données incohérentes avant qu'elles n'entrent.**

Modifier des lignes : UPDATE

```sql
UPDATE produit
SET    prix_ht = ROUND(prix_ht * 1.04, 2)
WHERE  matiere = 'aluminium' ;
```

Cinq lignes modifiées — les cinq références en aluminium :

ref_produit	prix_ht avant	prix_ht après
F-210	420,00	436,80
F-315	310,00	322,40
P-120	690,00	717,60
V-400	8 400,00	8 736,00
C-900	1 250,00	1 300,00

- Plusieurs colonnes se modifient dans le même `SET`, séparées par des virgules :

```sql
UPDATE client
SET    ville = 'Vendôme',
       categorie = 'collectivité'
WHERE  id_client = 8 ;
```

- La condition d'un `UPDATE` peut elle aussi contenir une sous-requête. Camille Vasseur accorde une remise de 2 % sur les chantiers **en cours** de ses **promoteurs**. La catégorie est dans `CLIENT`, la date de fin dans `CHANTIER` :

```sql
UPDATE chantier
SET    montant_ht = ROUND(montant_ht * 0.98, 2)
WHERE  date_fin IS NULL
AND    id_client IN (SELECT id_client
                     FROM   client
                     WHERE  categorie = 'promoteur') ;
```

Deux lignes modifiées :

id_chantier	montant_ht avant	montant_ht après
102	61 200,00	59 976,00
106	96 750,00	94 815,00

Les **deux** conditions sont indispensables. Sans `date_fin IS NULL`, la remise toucherait aussi les chantiers achevés des deux promoteurs (101 et 105). Sans la sous-requête, elle toucherait les chantiers en cours des particuliers (107 et 109).

⚠ L'erreur qui coûte cher

Un `UPDATE` sans `WHERE` modifie toutes les lignes de la table.

```
1  UPDATE produit SET prix_ht = prix_ht * 1.04 ;   -- les 8 références
augmentées
```

Aucun message, aucune confirmation : la requête réussit. C'est l'accident le plus fréquent, et le plus difficile à rattraper — le prix d'origine n'existe plus nulle part.

Le `WHERE` n'est pas facultatif dans la pratique : **il est facultatif pour le SGBD, jamais pour vous.**

Supprimer des lignes : DELETE FROM

```
1  DELETE FROM chantier
2  WHERE  date_fin IS NOT NULL
3     AND date_fin < '2020-01-01' ;
```

Sur cette base, la requête ne supprime **aucune ligne** : tous les chantiers sont postérieurs à 2020. Un `DELETE` qui ne supprime rien n'est pas une erreur — c'est un `DELETE` dont la condition n'a trouvé personne.

⚠ Les deux pièges de DELETE

- **`DELETE FROM produit ;` vide la table entière.** Même remarque que pour `UPDATE`, avec des conséquences pires : les lignes sont perdues ;
- **L'intégrité référentielle peut bloquer la suppression** — et c'est heureux. Supprimer le client 1 laisserait les chantiers 101 et 102 rattachés à un client inexistant. Le SGBD refuse, et il a raison : la réponse n'est pas de forcer, mais de se demander si l'on veut vraiment supprimer, ou plutôt **désactiver**.

Supprimer ou désactiver ?

En gestion, la suppression est presque toujours le mauvais réflexe. Un client qui ne commande plus n'a pas disparu : ses chantiers, ses factures et son historique doivent

rester justifiables — c'est une exigence de **piste d'audit** (voir 1.4).

La bonne pratique consiste à ajouter une colonne `actif` et à écrire `UPDATE client SET actif = 0 WHERE …` plutôt que `DELETE`. On parle de **suppression logique**, par opposition à la suppression physique.

Une image pour comprendre

`SELECT`, c'est consulter le dossier. `UPDATE`, c'est corriger une ligne au stylo. `DELETE`, c'est passer le dossier au broyeur.

On relit toujours avant de corriger. On relit deux fois avant de broyer. Et dans un cabinet comptable, on ne broie pratiquement jamais : on archive.

Chez Vasseur — la réponse

Comment modifie-t-on des données en masse, et comment ne pas détruire la base en le faisant ?

Les références en aluminium tiennent en une requête, et la soirée de Nadia Perrot est terminée :

```
UPDATE produit
SET    prix_ht = ROUND(prix_ht * 1.04, 2)
WHERE  matiere = 'aluminium' ;
```

Mais la requête ne s'écrit pas en premier. La **méthode** compte plus que la syntaxe, et elle tient en quatre gestes :

1. **Compter d'abord.** `SELECT COUNT(*) FROM produit WHERE matiere = 'aluminium' ;` doit renvoyer 5. Si le chiffre diffère, le `WHERE` est faux — et il aurait été faux dans l'`UPDATE` aussi ;
2. **Regarder ensuite.** Le même `WHERE` en `SELECT *` montre exactement les lignes qui vont bouger ;
3. **Remplacer le verbe.** `SELECT` devient `UPDATE`, le `WHERE` ne bouge plus ;
4. **Vérifier après.** Le SGBD annonce le nombre de lignes affectées : il doit dire 5.

Trois erreurs à ne jamais commettre, dans l'ordre de gravité : **un `UPDATE` sans `WHERE`** (toute la table), **un `DELETE` à la place d'un `UPDATE`** (irréversible), et **une modification en masse sans sauvegarde préalable** en production.

Et le point de gestion, qui dépasse la technique : cette augmentation de 4 % **écrase les anciens prix**. Si l'entreprise doit pouvoir justifier le prix appliqué à un chantier de mars, il ne faut pas modifier le tarif — il faut en **historiser** un nouveau, avec sa date d'effet. C'est très exactement la leçon du chapitre 2.2.

✓ **Vérifiez que vous avez compris**

Un collègue vous dit : « j'ai lancé un `UPDATE`, il a modifié 4 000 lignes au lieu de 12 ». Que s'est-il passé, et que fallait-il faire ?

Son `WHERE` était trop large — ou absent. La requête a fait exactement ce qu'on lui demandait ; c'est la condition qui ne décrivait pas les 12 lignes voulues.

Il fallait **écrire la requête en `SELECT` d'abord** : `SELECT * FROM … WHERE …` aurait affiché 4 000 lignes, et l'erreur aurait sauté aux yeux **avant** toute modification.

Et maintenant ? Si une transaction est ouverte, `ROLLBACK`. Sinon, seule une **sauvegarde** permet de revenir en arrière — d'où l'importance de la fonction « sécuriser » du SI, vue en 1.1, qui cesse d'être une abstraction le jour où l'on en a besoin.

Votre tour

Ici, allez-y franchement : c'est la seule section où l'erreur est instructive.

1. augmentez de 10 % le prix des produits en PVC, puis vérifiez le résultat ;
2. lancez volontairement un `UPDATE` **sans** `WHERE` et regardez le nombre de lignes affectées ;
3. réinitialisez la base, puis recommencez proprement.

Mise en pratique

Cas pratique 4 : Ce que les poseurs font remonter

Compétences attendues

- Écrire des requêtes de mise à jour de données

Sur la base de Menuiseries Vasseur. Réinitialisez la base, puis traitez les questions **dans l'ordre** : chacune part de l'état laissé par la précédente.

1. **Écrivez la requête qui enregistre une nouvelle salariée** : Inès Garnier, poseuse, site de Tours, embauchée le 4 mai 2026. Le numéro de salarié suivant est le 8.
2. **Inès Garnier a travaillé 6,5 heures sur le chantier 107 le 5 mai 2026. Enregistrez cette intervention.** Que se serait-il passé si vous aviez traité la question 2 avant la question 1 ?
3. **L'intervention de Léa Aubry du 18 février 2026 sur le chantier 110 a duré 5,5 heures, et non 5. Corrigez-la.** Justifiez chacune des conditions de votre `WHERE`.
4. **Les chantiers en cours de Delaunay Hervé sont terminés au 30 avril 2026. Renseignez leur date de fin**, sans utiliser le numéro du client ni celui du chantier.
5. **Écrivez une requête qui supprime le produit C-900, puis une autre qui supprime le produit F-210. L'une est acceptée, l'autre refusée. Expliquez pourquoi.**
6. **Yann Mercier quitte l'entreprise. Faut-il le supprimer de la base ? Justifiez votre réponse, et proposez ce qu'il faudrait faire à la place.**
7. **Appliquez à la requête de la question 4 la méthode en quatre gestes vue dans le cours.**

Vérifiez vos réponses

La base est **exécutable** : ses données sont sur la page **La base de données Menuiseries Vasseur**. Écrivez vos requêtes ci-dessous et comparez.

Synthèse

Le format d'une requête d'extraction de données (interrogation) sans imbrication

Les clauses s'écrivent **toujours dans cet ordre**. Toutes sont facultatives sauf `SELECT` et `FROM`.

Clause et rôle	Exemple sur la base Vasseur
`SELECT [DISTINCT] champ1, champ2…` **Projection** — quelles colonnes	`SELECT c.categorie, SUM(ch.montant_ht) AS ca_ht`
`FROM table T` La table de départ	`FROM chantier ch`
`INNER JOIN … ON` **Jointure** — quelles tables rapprocher, et comment	`INNER JOIN client c ON ch.id_client = c.id_client`
`WHERE prédicat` **Restriction** — quelles lignes de détail	`WHERE ch.date_fin IS NOT NULL`
`GROUP BY champ1, champ2` **Regroupement**	`GROUP BY c.categorie`
`HAVING prédicat agrégé` **Restriction sur les groupes**	`HAVING SUM(ch.montant_ht) > 50000`
`ORDER BY champ [ASC / DESC]` **Tri**	`ORDER BY ca_ht DESC`

Soit, d'un seul tenant :

```
SELECT     c.categorie, COUNT(*) AS nb_chantiers, SUM(ch.montant_ht) AS ca_ht
FROM       chantier ch
INNER JOIN client c ON ch.id_client = c.id_client
WHERE      ch.date_fin IS NOT NULL
GROUP BY   c.categorie
HAVING     SUM(ch.montant_ht) > 50000
ORDER BY   ca_ht DESC ;
```

Avec imbrication : la sous-requête

Forme	Quand l'employer	Exemple sur la base Vasseur
`WHERE champ > (SELECT agrégat …)`	Comparer chaque ligne à un chiffre calculé : moyenne, maximum…	`WHERE montant_ht > (SELECT AVG(montant_ht) FROM chantier)`
`WHERE champ IN (SELECT …)`	Garder les lignes qui figurent dans une liste calculée	`WHERE id_client IN (SELECT id_client FROM chantier WHERE date_fin IS NULL)`
`WHERE champ NOT IN (SELECT …)`	Garder celles qui n'y figurent pas	`WHERE ref_produit NOT IN (SELECT ref_produit FROM poser)`

Les trois distinctions qui font la différence à l'examen

- **`WHERE` filtre des lignes, `HAVING` filtre des groupes.** Toute condition portant sur `SUM`, `COUNT`, `AVG`, `MIN`, `MAX` va dans le `HAVING` ;
- **`ON` rapproche, `WHERE` sélectionne.** Ne mettez jamais une condition de jointure dans le `WHERE` ;
- **`INNER JOIN` perd les lignes sans correspondance, `LEFT JOIN` les conserve.** Un « et ceux qui n'en ont pas » dans l'énoncé impose une jointure externe.

Les jointures

Jointure	Ce qu'elle garde
`INNER JOIN`	Uniquement les lignes appariées des deux côtés
`LEFT OUTER JOIN`	Toutes les lignes de la table de gauche, appariées ou non
`RIGHT OUTER JOIN`	Toutes les lignes de la table de droite, appariées ou non
`FULL OUTER JOIN`	Toutes les lignes des deux tables

Jointure	Ce qu'elle garde
`NATURAL JOIN`	Jointure implicite sur les colonnes de même nom — **à éviter** : elle dépend du nommage et casse dès qu'une colonne est ajoutée
`UNION`	Ce n'est pas une jointure : elle empile les lignes de deux requêtes de même schéma. `UNION` dédoublonne, `UNION ALL` non

Les opérateurs du `WHERE`

`=` · `<>` · `<` · `>` · `<=` · `>=` · `BETWEEN … AND …` · `IN (…)` · `LIKE 'A%'` · `IS NULL` · `IS NOT NULL` — combinés par `AND`, `OR`, `NOT`.

> ⚠ **`NULL` ne se compare pas**
>
> `WHERE date_fin = NULL` ne renvoie **jamais rien**. Il faut `IS NULL` ou `IS NOT NULL`. C'est l'une des erreurs les plus fréquentes, et elle ne provoque pas de message : elle renvoie un résultat vide.

Le format des requêtes de mise à jour des données

	`INSERT`	`UPDATE`	`DELETE`
Syntaxe	`INSERT INTO Table (champ1, champ2…) VALUES (val1, val2…) ;`	`UPDATE Table SET colonne = valeur WHERE condition ;`	`DELETE FROM Table WHERE condition ;`
Sans `WHERE`	Sans objet	**Modifie toute la table**	**Vide toute la table**
Réversible ?	Par un `DELETE` ciblé	Non — l'ancienne valeur est perdue	Non — hors sauvegarde ou transaction

⚠ **La méthode, en quatre gestes**

1. **Compter** : `SELECT COUNT(*) FROM … WHERE …` — le chiffre est-il celui attendu ?
2. **Regarder** : le même `WHERE` en `SELECT *` — sont-ce les bonnes lignes ?
3. **Remplacer le verbe** : `SELECT` devient `UPDATE` ou `DELETE`, le `WHERE` ne bouge plus ;
4. **Vérifier** : le nombre de lignes affectées annoncé par le SGBD doit correspondre.

Et en gestion : préférez presque toujours la **suppression logique** (`UPDATE … SET actif = 0`) à la suppression physique, pour préserver la piste d'audit.

◀ 2.2 Le modèle relationnel avancé · 2.4 Le tableur : généralités ▶